T2

Support Vector Machines

Salvador Ruiz Correa

November 5th, 2026

A Support Vector Machineis a supervised machine learning algorithm used for classification and regression tasks. It works by finding the optimal hyperplane that separates data points of different classes with the maximum margin. Key features of SVM include:

- Maximizing the Margin: SVM aims to find the hyperplane that maximizes the distance (margin) between the closest data points of different classes, known as support vectors.
- Kernel Trick: For non-linearly separable data, SVM uses kernel functions (e.g., linear, polynomial, or RBF) to map the input data into a higher-dimensional space where a linear separation is possible.
- Soft Margin: In real-world scenarios where data is not perfectly separable, SVM introduces slack variables to allow for some misclassifications, controlled by a regularization parameter C.
- Versatility: SVM is effective in high-dimensional spaces and can handle both linear and non-linear data, making it suitable for applications like image classification, text categorization, and bioinformatics.
- Binary and Multi-class Classification: While SVM is inherently a binary classifier, it can be extended to multi-class problems using strategies like One-vs-One or One-vs-Rest.

Agenda:
1 Support Vector Machine.
2 Kernel Trick.
3 Multiclasss Classification.
4 Bagging, Boosting, Stacking.

Introduction

A Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification and regression tasks. It is particularly effective in high-dimensional spaces and is widely used in various applications such as image classification, text categorization, and bioinformatics.

Mathematical Formulation

Given a set of training examples $\{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$, where $x_i \in \mathbb{R}^d$ and $y_i \in \{-1, 1\}$, the goal of SVM is to find the hyperplane that maximizes the margin between the two classes.

The hyperplane can be defined as:

$$w \cdot x + b = 0$$

where w is the weight vector and b is the bias term.

The optimization problem for SVM can be formulated as:

$$\min_{w,b} \frac{1}{2}\|w\|^2 \quad \text{subject to} \quad y_i(w \cdot x_i + b) \geq 1 \quad \forall i$$

Kernel Trick

In cases where the data is not linearly separable, SVM can use the kernel trick to map the input data into a higher-dimensional space where a linear separation is possible. Common kernel functions include:

- Linear Kernel: $K(x_i, x_j) = x_i \cdot x_j$
- Polynomial Kernel: $K(x_i, x_j) = (x_i \cdot x_j + 1)^d$
- Radial Basis Function (RBF) Kernel: $K(x_i, x_j) = \exp(-\gamma\|x_i - x_j\|^2)$

Examples

Example 1: Linear SVM

Consider a simple 2D dataset with two classes. The SVM algorithm finds the optimal hyperplane that separates the two classes with the maximum margin.

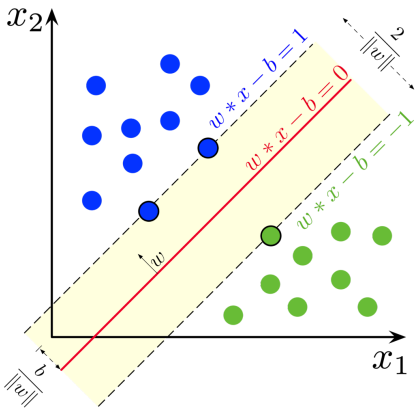


Figure 1: Linear SVM separating two classes in 2D space.

Example 2: Non-linear SVM

The RBF kernel can map the data into a higher-dimensional space where a linear separation is possible for a non-linearly separable dataset.

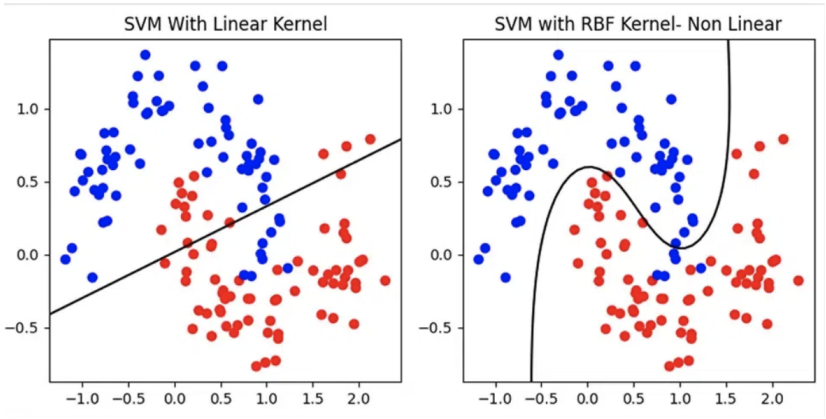


Figure 2: Non-linear SVM using RBF kernel.

Technical Details

Soft Margin SVM

In real-world scenarios, data may not be perfectly separable. Soft margin SVM introduces slack variables ξ_i to allow for misclassification:

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i \quad \text{subject to} \quad y_i(w\cdot x_i + b) \geq 1 - \xi_i \quad \text{and} \quad \xi_i \geq 0 \quad \forall i$$

where C is the regularization parameter.

Multi-class SVM

SVM is inherently a binary classifier. For multi-class classification, strategies like One-vs-One or One-vs-Rest are used.

Introduction

Support Vector Machines (SVMs) are inherently designed for binary classification. However, many real-world problems involve multi-class classification. To extend SVMs to multi-class problems, two common strategies are used: **One-vs-Rest (OvR)** and **One-vs-One (OvO)**. This document explains both strategies in detail.

One-vs-Rest (OvR)

Overview

In the **One-vs-Rest** strategy, one binary SVM classifier is trained for each class. Each classifier is trained to distinguish one class from all

the other classes combined.

Mathematical Formulation

For N classes, N binary SVM classifiers are trained. For the i-th class, the classifier is trained with:

Positive examples: Class iNegative examples: All classes except i

The decision function for the i-th classifier is:

$$f_i(x) = w_i \cdot x + b_i$$

During prediction, the class with the highest score is selected:

$$\text{Predicted class} = \arg\max_i f_i(x)$$

Example

Suppose there are 3 classes: **A**, **B**, and **C**.
- Classifier 1: **A** vs **B+C**
- Classifier 2: **B** vs **A+C**
- Classifier 3: **C** vs **A+B**

For a new data point, each classifier outputs a score, and the class with the highest score is selected.

Pros and Cons

- **Pros**:
  - Simple and computationally efficient for a small number of classes.
  - Requires training only N classifiers.
- **Cons**:
  - May struggle with imbalanced datasets.
  - Overlapping classes can reduce performance.

One-vs-One (OvO)

Overview

In the **One-vs-One** strategy, a binary SVM classifier is trained for every pair of classes. For N classes, this results in $\frac{N(N-1)}{2}$ classifiers.

Mathematical Formulation

For each pair of classes (i, j), a binary SVM classifier is trained:

$$f_{ij}(x) = w_{ij} \cdot x + b_{ij}$$

During prediction, each classifier votes for one of the two classes it was trained on. The class with the most votes is selected:

$$\text{Predicted class} = \arg\max_i \sum_{j \neq i} \text{votes}(f_{ij}(x))$$

Example

Suppose there are 3 classes: **A**, **B**, and **C**.
- Classifier 1: **A** vs **B**
- Classifier 2: **A** vs **C**
- Classifier 3: **B** vs **C**

For a new data point, each classifier votes for one of the two classes, and the class with the most votes is selected.

Pros and Cons

- **Pros**:
  - Handles imbalanced datasets better.
  - Often achieves better performance for complex datasets.
- **Cons**:
  - Computationally expensive for a large number of classes.
  - Requires more memory and prediction time.

Comparison

Aspect	**One-vs-Rest (OvR)**	**One-vs-One (OvO)**
Number of Classifiers	N	$\frac{N(N-1)}{2}$
Training Data	All data for each classifier	Data from two classes per classifier
Scalability	Better for large N	Better for small N
Imbalanced Data	May struggle	Handles better
Prediction Time	Faster	Slower

Both **One-vs-Rest** and **One-vs-One** are effective strategies for extending SVMs to multi-class classification. The choice between them depends on the number of classes, dataset characteristics, and computational resources. For large datasets with many classes, **OvR** is often preferred due to its efficiency, while **OvO** is better suited for smaller datasets or when handling imbalanced classes.

Support Vector Machines for Regression (SVR)

Support Vector Machines (SVMs) are a powerful and versatile class of supervised learning algorithms commonly used for both classifi-

cation and regression tasks. While SVMs are widely known for their application in classification, they can also be adapted for regression problems, where the goal is to predict a continuous output variable. This adaptation is known as **Support Vector Regression (SVR)**.

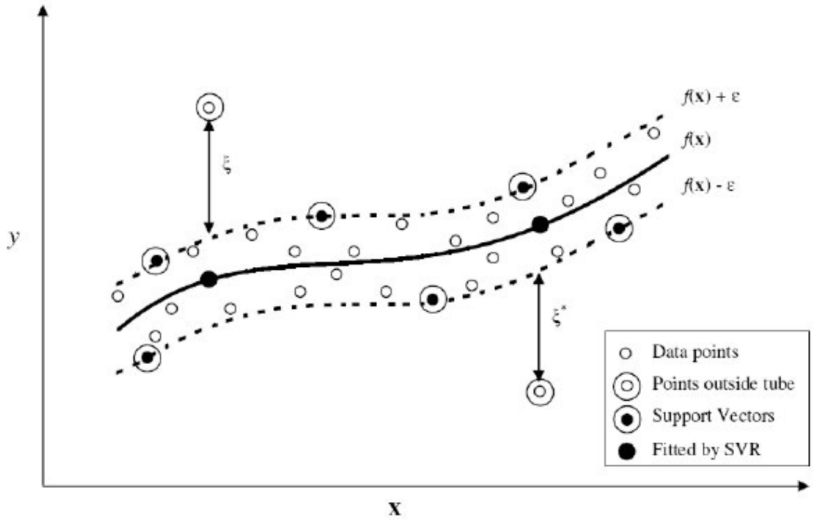


Figure 3: SVR.

Key Idea of SVR

The core idea of SVR is to find a function $f(x)$ that approximates the relationship between input features x and a continuous target variable y, while ensuring that the deviation between the predicted and actual values is within a specified tolerance ϵ. Unlike traditional regression methods that minimize the squared error, SVR focuses on minimizing the **epsilon-insensitive loss**, which ignores errors smaller than ϵ.

Mathematical Formulation

1. **Objective**:
   SVR seeks to find a linear regression function:

$$f(x) = w^T x + b$$

where:
- w is the weight vector,
- b is the bias term,
- x is the input feature vector.

2. **Epsilon-Insensitive Loss**:

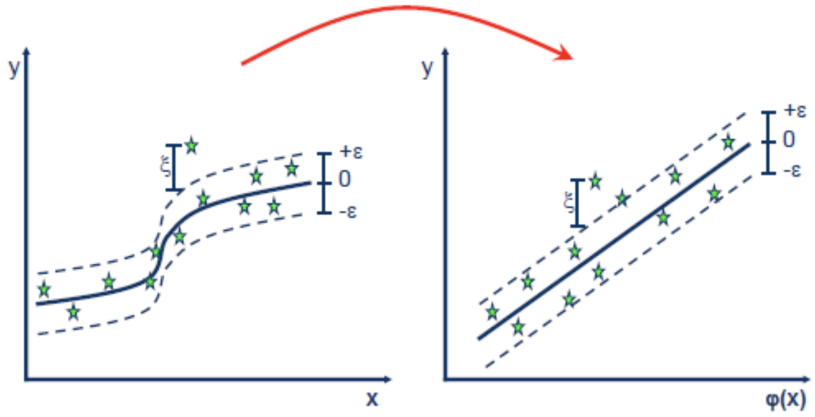


Figure 4: SVR.

The loss function in SVR is defined as:

$$L_\epsilon(y, f(x)) = \begin{cases} 0 & \text{if } |y - f(x)| \leq \epsilon, \\ |y - f(x)| - \epsilon & \text{otherwise.} \end{cases}$$

This means that errors within ϵ are ignored, and only deviations larger than ϵ are penalized.

3. **Optimization Problem**:

The goal of SVR is to minimize the following objective function:

$$\frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}(\xi_i + \xi_i^*)$$

subject to:

$$\begin{cases} y_i - (w^T x_i + b) \leq \epsilon + \xi_i, \\ (w^T x_i + b) - y_i \leq \epsilon + \xi_i^*, \\ \xi_i, \xi_i^* \geq 0, \end{cases}$$

where:

- C is a regularization parameter that controls the trade-off between model complexity and training error,
- ξ_i and ξ_i^* are slack variables that allow for deviations outside the ϵ-tube.

Kernel Trick for Nonlinear Regression

For nonlinear regression problems, SVR can be extended using the **kernel trick**. Instead of working in the original feature space, the data is mapped to a higher-dimensional space using a kernel function $K(x_i, x_j)$. Common kernel functions include:

- Linear kernel: $K(x_i, x_j) = x_i^T x_j$,
- Polynomial kernel: $K(x_i, x_j) = (x_i^T x_j + c)^d$,

- Radial Basis Function (RBF) kernel: $K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$.

Advantages of SVR

- Robust to outliers due to the ϵ-insensitive loss function.
- Flexible and can model nonlinear relationships using kernel functions.
- Provides a unique solution due to the convex optimization problem.

Disadvantages of SVR

- Computationally expensive for large datasets.
- Requires careful tuning of hyperparameters (C, ϵ, and kernel parameters).

Ensemble Classifiers

Ensemble classifiers combine the predictions of multiple base classifiers to improve overall performance. The idea is that by aggregating the outputs of several models, the ensemble can reduce variance, bias, and generalization error. Common ensemble methods include **bagging**, **boosting**, and **stacking**.

Key Concepts

Bias-Variance Tradeoff

Ensemble methods aim to balance the bias-variance tradeoff:

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

- **Bias**: Error due to overly simplistic assumptions in the model.
- **Variance**: Error due to the model's sensitivity to small fluctuations in the training set.
- **Irreducible Error**: Noise inherent in the data.

Ensembles reduce variance by averaging multiple models (e.g., bagging) or reduce bias by sequentially improving models (e.g., boosting).

Diversity

The effectiveness of an ensemble depends on the **diversity** of its base classifiers. Diverse classifiers make different errors, and their combination can correct individual mistakes.

Ensemble Methods

Bagging (Bootstrap Aggregating)

Bagging reduces variance by training multiple base classifiers on different subsets of the training data and averaging their predictions.

Algorithm

1. Generate B bootstrap samples from the training data. 2. Train a base classifier $f_b(x)$ on each bootstrap sample. 3. Combine predictions by averaging (for regression) or majority voting (for classification):

$$f_{\text{bagged}}(x) = \frac{1}{B} \sum_{b=1}^{B} f_b(x) \quad \text{(regression)}$$

$$f_{\text{bagged}}(x) = \arg\max_c \sum_{b=1}^{B} \mathbb{I}(f_b(x) = c) \quad \text{(classification)}$$

Example

Random Forest is a popular bagging method that uses decision trees as base classifiers and introduces additional randomness by selecting a random subset of features at each split.

Boosting

Boosting reduces bias by sequentially training base classifiers, where each new classifier focuses on correcting the errors of the previous ones.

Algorithm

1. Initialize weights for each training example: $w_i = \frac{1}{N}$. 2. For $b = 1$ to B:

- Train a base classifier $f_b(x)$ on the weighted training data.
- Compute the weighted error: $\epsilon_b = \sum_{i=1}^{N} w_i \mathbb{I}(f_b(x_i) \neq y_i)$.
- Compute the classifier weight: $\alpha_b = \frac{1}{2} \ln\left(\frac{1-\epsilon_b}{\epsilon_b}\right)$.
- Update weights: $w_i \leftarrow w_i \exp(-\alpha_b y_i f_b(x_i))$ and renormalize.

3. Combine predictions using weighted voting:

$$f_{\text{boosted}}(x) = \text{sign}\left(\sum_{b=1}^{B} \alpha_b f_b(x)\right)$$

Example

AdaBoost and **Gradient Boosting** are popular boosting methods. Gradient Boosting uses gradient descent to minimize a loss function, while AdaBoost focuses on misclassified examples.

Stacking

Stacking combines multiple base classifiers using a meta-classifier, which learns how to best aggregate the base models' predictions.

Algorithm

1. Train B base classifiers $f_1(x), f_2(x), \ldots, f_B(x)$ on the training data.
2. Generate predictions for each base classifier on a validation set.
3. Train a meta-classifier g on the base classifiers' predictions.
4. Combine predictions:

$$f_{\text{stacked}}(x) = g(f_1(x), f_2(x), \ldots, f_B(x))$$

Example

A common approach uses logistic regression or a neural network as the meta-classifier.

Advantages of Ensemble Methods

- **Improved Accuracy**: Ensembles often outperform individual classifiers by reducing bias and variance.
- **Robustness**: Ensembles are less sensitive to noise and overfitting.
- **Flexibility**: Can combine different types of base classifiers.

Challenges

- **Computational Cost**: Training multiple models can be computationally expensive.
- **Interpretability**: Ensembles are often harder to interpret than individual models.
- **Overfitting Risk**: Complex ensembles may overfit if not properly regularized.

Ensemble classifiers are powerful tools for improving predictive performance by combining the strengths of multiple base models. Methods like bagging, boosting, and stacking offer different ways to achieve this, each with its advantages and challenges. The choice of ensemble method depends on the problem, dataset, and computational resources.



Figure 5: Stacking.